

Объектно-ориентированное программирование

Никита Остроухов

Кафедра информатики МФТИ

Работа со строками

Строки в C++

```
#include <iostream>

int main()
{
    char* str = "My string";

    ...
}
```

Строкой является массив **char** элементов

Строки в C++

```
#include <iostream>

int main()
{
    char* str = "My string";

    std::cout << str << std::endl;

    ...
}
```

Если мы выведем строку на экране то
напечатается:

My string

Строки в C++

```
#include <iostream>

int main()
{
    char* str = "My string";

    std::cout << str << std::endl;
    ...
}
```



Массив не умеет хранить свой размер

Как **std::cout** определил размер массива char?

Строки в C++

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    char* str = "My string";
```

```
    ...
```



Наша строка **состоит** из 9 символов

Строки в C++

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    char* str = "My string";
```

```
    ...
```



Наша строка **состоит** из 9 символов

Но **размер** массива равен 10 элементам

Устройство строк

			M	y		s	t	r	i	n	g								
--	--	--	---	---	--	---	---	---	---	---	---	--	--	--	--	--	--	--	--

Массив в памяти представлен его
элементами

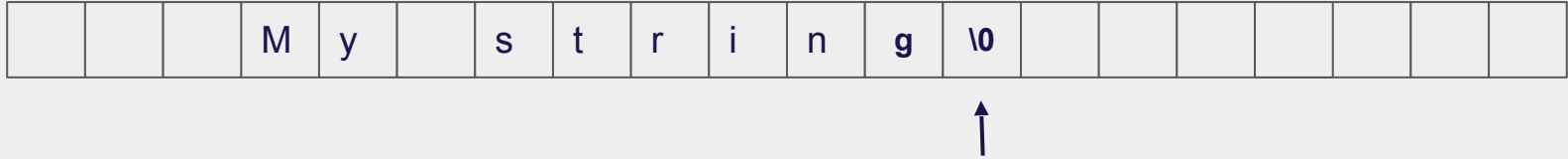
Устройство строк

			M	y		s	t	r	i	n	g							
--	--	--	---	---	--	---	---	---	---	---	---	--	--	--	--	--	--	--

```
char* str = "My string";
```

Создавая строку в C++ дополнительным
элементам в массиве является
символ окончания строки

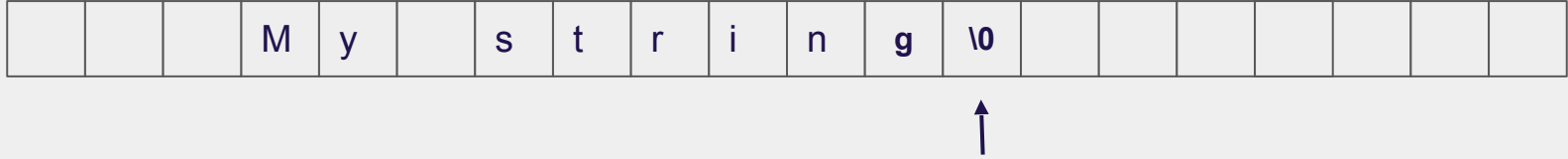
Устройство строк



```
char* str = "My string";
```

Создавая строку в C++ дополнительным
элементам в массиве является
символ окончания строки

Устройство строк



```
char* str = "My string";
```

\0 - служит **признаком** окончания строки

Задание

Попробуйте затереть **символ** `\0` в конце строки
и напечатать ее

Задание

Реализовать библиотеку для работы со строками

Библиотека строк

```
int strlen(char* str)
```

Функция возвращающая длину строки

Библиотека строк

```
int strlen(char* str)  
int countCharInString(char* str, char c)
```

Подсчет количества символа **c** в строке **str**

Библиотека строк

```
int strlen(char* str)
int countCharInString(char* str, char c)
char* mergeStrings(char* str1, char* str2)
```

Слияние двух строк

Библиотека строк

```
int strlen(char* str)
int countCharInString(char* str, char c)
char* mergeStrings(char* str1, char* str2)
char** cutString(char* str, char delimiter)
```

Получение массива строк разделенных
символами **delimiter**

Работа с файлами

Потоки ввода и вывода

```
#include <iostream>

int main()
{
    std::cout << "Print anything" << std::endl;
    std::cin >> anything;

    ...
}
```

**Мы научились печатать и получать
информацию с консоли**

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::cout << "Print anything" << std::endl;
    std::cin >> anything;

    std::ofstream file;
    file.open ("example.txt");

    file << "Write anything" << std::endl;
    file.close();

    ...
}
```

С файлами делать это также просто

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::cout << "Print anything" << std::endl;
    std::cin >> anything;

    std::ofstream file;
    file.open ("example.txt");

    file << "Write anything" << std::endl;
    file.close();

    ...
}
```

Создание **потока вывода** в файл

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::cout << "Print anything" << std::endl;
    std::cin >> anything;

    std::ofstream file;
    file.open ("example.txt");

    file << "Write anything" << std::endl;
    file.close();

    ...
}
```

Связывание **потока вывода** с конкретным файлом файловой системы

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::cout << "Print anything" << std::endl;
    std::cin >> anything;

    std::ofstream file;
    file.open ("example.txt");

    file << "Write anything" << std::endl;
    file.close();

    ...
}
```

**Запись строки в поток и синхронизация
буфера обмена**

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::cout << "Print anything" << std::endl;
    std::cin >> anything;

    std::ofstream file;
    file.open ("example.txt");

    file << "Write anything" << std::endl;
    file.close();
    ...
}
```



Заккрытие файла

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::ofstream file;
    file.open ("example.txt");

    file << "Write anything" << std::endl;
    file.close();

    ...
}
```

Запись в файл

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::ofstream file("example.txt");

    file << "Write anything" << std::endl;
    file.close();

    ...
}
```

Запись в файл

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");

    char text[100];

    file >> text;
    file.close();

    ...
}
```

Чтение из файла

Потоки ввода и вывода

```
#include <iostream>
#include <fstream>
```

```
int main()
{
```

```
    std::ifstream file("example.txt");
```

```
    char text[100];
```

Поток ввода ifstream

```
    file >> text;
    file.close();
```

```
    ...
```

Чтение из файла

Проблемы ifstream

```
#include <iostream>
#include <fstream>
```

```
int main()
{
```

```
    std::ifstream file("example.txt");
```

```
    char text[100];
```

```
    file >> text;
    file.close();
```

```
    ...
```

**Считает только
первое слово**



Чтение из файла

Проблемы ifstream

```
#include <iostream>
#include <fstream>
```

```
int main()
{
```

```
    std::ifstream file("example.txt");
```

```
    char text[100];
```

```
    file >> text;
    file.close();
```

```
    ...
```

←
**Нет гарантий что мы не
выйдем за пределы
массива**

Чтение из файла

read и write

Чтение через read

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");

    char text[100];

    file.read(text, 100);
    file.close();

    ...
}
```

Функция read считывает заданное количество символов из файла

Чтение через read

```
#include <iostream>
#include <fstream>
```

```
int main()
{
    std::ifstream file("example.txt");
```

```
    char text[100];
```

```
    file.read(text, 100);
    file.close();
```

```
    ...
```

Количество символов



Функция **read** считывает заданное количество символов из файла

Запись через write

```
#include <iostream>
#include <fstream>
```

```
int main()
{
    std::ofstream file("example.txt");

    char text[100];

    file.write(text, 100);
    file.close();

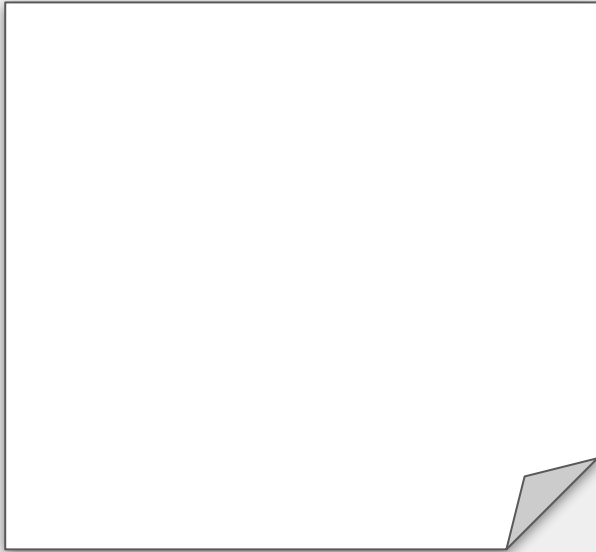
    ...
}
```

Функция **write** записывает заданное количество символов в файл

Как вычислить длину файла?

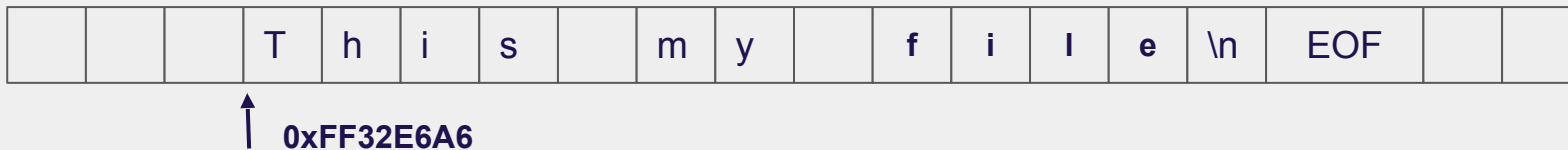
Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--



Представление **файла**
в файловой системе

Файловая система



```
std::ifstream file("example.txt");
```

Открывая файл мы получаем
его **дескриптор**

Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--



0xFF32E6A6



Число ассоциируемое с
областью

```
std::ifstream file("example.txt");
```

Открывая файл мы получаем
его дескриптор

Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--

↑ 0xFF32E6A6

```
std::ifstream file("example.txt");
```

Переменная содержащая
дескриптор

Число ассоциируемое с
областью

Открывая файл мы получаем
его дескриптор

Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--



```
std::ifstream file("example.txt");
```

EOF специальный символ
означающий **конец** файла

Файловая система



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

Считывая файл мы перемещаем
файловую **каретку**

Файловая система



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

Считывая файл мы перемещаем
файловую **каретку**

Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

Каретка указывает на **текущий символ**
для чтения в файле

Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

```
int len = file.tellg();
```

Функция **tellg** возвращает расстояние
от начала файла
до текущей позиции каретки

Файловая система

			T	h	i	s		m	y		f	i	l	e	\n	EOF		
--	--	--	---	---	---	---	--	---	---	--	---	---	---	---	----	-----	--	--



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

```
int len = file.tellg();
```

Функция **tellg** возвращает расстояние
от начала файла
до текущей позиции каретки

Файловая система



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

```
file.seekg(0, std::ios::end);  
int len = file.tellg();
```

Функция **seekg** может изменять
положение каретки в файле

Файловая система



```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

```
file.seekg(0, std::ios::end);  
int len = file.tellg();
```

seekg выставил положение каретки в
std::ios::end (конец файла)
со смещением 0

Файловая система



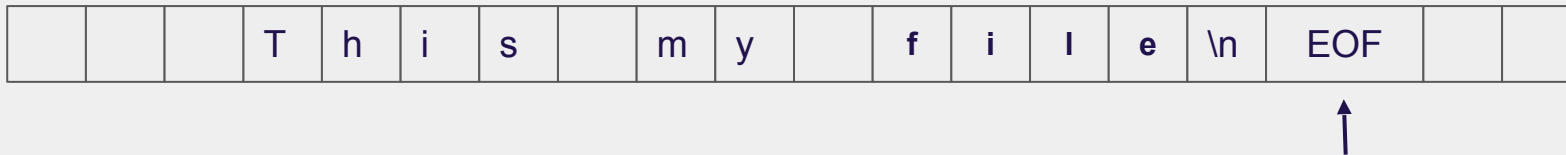
```
std::ifstream file("example.txt");
```

```
char text[100];  
file.read(text, 5);
```

```
file.seekg(0, std::ios::end);  
int len = file.tellg();
```

Переменная **len** хранит теперь длину
файла

Файловая система



```
int fileLen(std::ifstream* fd)
{
    fd -> seekg(0, std::ios::end);
    int len = file.tellg();

    return len;
}
```

Простейшая функция вычисления
длины файла

Файловая система



```
int fileLen(std::ifstream* fd)
{
    fd -> seekg(0, std::ios::end);
    int len = file.tellg();
    fd -> seekg(0, std::ios::beg);

    return len;
}
```

Нужно не забыть поставить каретку в
начало файла после
завершения работы

Чтение через read

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");
```

Чтение файла с произвольной длиной

Чтение через read

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");
    int len = fileLen(&file)
```

Чтение файла с произвольной длиной

Чтение через read

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");
    int len = fileLen(&file)

    char buffer = new char[len];
```

Чтение файла с произвольной длиной

Чтение через read

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");
    int len = fileLen(&file)

    char buffer = new char[len];

    file.read(buffer, len);
    file.close();

    ...
}
```

Чтение файла с произвольной длиной

Чтение через read

```
#include <iostream>
#include <fstream>

int main()
{
    std::ifstream file("example.txt");
    int len = fileLen(&file)

    char buffer = new char[len];

    file.read(buffer, len);
    file.close();

    ...

    delete[] buffer;
```

Чтение файла с произвольной длиной

Задание

**Используя графику и события в SFML
реализовать HackerTyper**

www.hackertyper.com